SISTEMA DE GESTIÓN ACADÉMICA – DECISIONES DE DISEÑO

YEIMMY JOHANA JIMENEZ YAYA

UNIVERSIDAD DE MANIZALES
FACULTAD DE CIENCIAS E INGENIERÍA
2026

1. Introducción

En este documento explico las decisiones que tomé para mejorar mi Sistema de Gestión Académica. En esta actividad hice que Persona fuera abstracta, agregué los grupos con su horario y creé excepciones para controlar errores.

2. Descripción general

El sistema maneja personas (estudiantes, docentes y administrativos), programas, asignaturas y grupos. Cada grupo tiene una asignatura, un docente, un horario y estudiantes, con un cupo máximo. Está hecho en Python con programación orientada a objetos.

3. Desarrollo

3.1 ¿Por qué Persona es abstracta?

Porque en el sistema nunca hay una persona sola: siempre es un estudiante, un docente o un administrativo. Persona solo guarda lo que tienen en común, y no se pueden crear objetos de ella. Si se intenta, Python da un error, y en main.py lo capturo para que el programa siga.

3.2 Método abstracto

realizar_actividad_principal() es abstracto en Persona. Cada clase hija lo escribe a su manera:

- **Estudiante:** dice qué carrera cursa y en qué semestre está.
- **Docente:** dice en qué facultad da clases.
- **Administrativo:** dice cuál es su cargo y su dependencia.

3.3 Relaciones del grupo

- El grupo recibe una asignatura y un docente que ya existen. Siguen existiendo, aunque el grupo no.
- El grupo guarda una lista de estudiantes.
- El grupo crea su propio horario.

3.4 ¿Por qué composición y no herencia?

Un grupo tiene un horario, pero no es un horario. Además, el horario solo sirve dentro de un grupo, por eso el grupo lo crea. La herencia se usa cuando algo es un tipo de otra cosa, como un estudiante que es una persona.

3.5 Excepciones

Hice cinco excepciones, todas en excepciones.py:

- **Estudiante ya matriculado:** si ya está en el grupo.
- **Grupo lleno:** si ya no hay cupos.
- **Retiro de no matriculado:** si se quiere retirar a alguien que no está en el grupo.
- **Asignatura no encontrada:** si el código no existe.
- **Programa no encontrado:** si el código no existe.

Se lanzan con raise donde se detecta el problema y se capturan con try/except en main.py. Así sale un mensaje claro y el programa no se detiene.

3.6 Polimorfismo

En main.py hay una lista con estudiantes, docentes y administrativos. La recorro con un solo for y llamo los mismos métodos. Cada objeto responde según su tipo, sin usar if.

4. Conclusiones

Aprendí que una clase abstracta obliga a las clases hijas a tener ciertos métodos, y que la composición se usa cuando una clase tiene un objeto de otra. También vi que las excepciones ayudan a que los errores se entiendan y el programa no se caiga.

5. Declaración de uso de IA

Usé Claude (una herramienta de IA) como apoyo en las dos actividades para entender conceptos de POO, armar la base del código, organizar este documento y resolver errores de Git. Revisé y ejecuté el código, y cambié mensajes y datos para dejarlos a mi manera.